Cloud auth & identity: proving who you are without a stored secret

> ***How to read this file:*** *the first half is GENERAL auth/identity knowledge (transferable to any cloud/job/interview — AWS, GCP, Azure, Kubernetes). The second half (marked "WORKED EXAMPLE") is how the WCX repo instantiates it. Learn the general model; use the repo to make it concrete.* ***Trust status:*** *conceptually solid (Day 6, ~5/6 quiz), read against credential.py / keyvault.py. The MI token mechanism (Entra issues+signs, not the resource) and the OBO cache-key rule are the two load-bearing correctness facts — both understood precisely. File:line details NOT re-verified 2026-07-28; re-grep* `get_azure_credential` */* `get_obo_credential` */* `NonClosingCredential` *before citing exact lines. General concepts are stable industry knowledge.* ***Related:*** *[[kb-rag-retrieval]] (* `get_azure_credential` *sits under every client), [[kb-storage-model]] (what the tokens unlock), [[kb-distributed-systems-patterns]] (caching, trust boundaries, verify-don't-trust). [[kb-harness-engineering]] (the capstone — auth guards L2 tool executors).*

PART 0 — WHY THIS MATTERS (philosophy & real-world connection)

Deep principle in one line: the most secure secret is the one that doesn't exist — derive identity from BEING the workload (platform-vouched) rather than from holding a stealable credential.

Why it exists / what it solves. The whole field circles one root problem: **secret zero.** Every credential you store is a permanent liability — it can leak, get committed, and must be rotated forever. Workload identity kills secret zero outright: your code holds *no* long-lived secret, so there's nothing to steal. This "stop storing secrets" shift is, in my view, the single biggest change in modern cloud security practice — everything else (vaults, short-lived tokens, mTLS) is scaffolding around it.

Real-world connection (beyond this repo). This is *everywhere*, not a WCX quirk. You meet it in every cloud app: AWS IAM roles, GCP workload identity, Azure Managed Identity, Kubernetes service accounts — same idea, four dialects. You meet it every time you "log in with Google" (OAuth/OIDC), every time an API hands back a token, every time an API validates a JWT (which is nearly all of them). Secret managers (HashiCorp Vault, Azure Key Vault, AWS Secrets Manager) are now table stakes, and service-to-service calls increasingly ride mTLS. Auth is simultaneously a top source of real-world breaches *and* one of the densest interview topics — so "never put a secret in code or an env var" is a rule that pays off for the whole career, not just this job.

What breaks without it (concrete failure modes):

- Committing secrets to git → breaches — the classic "AWS keys pushed to a public GitHub repo, bill in the thousands by morning" story.
- Treating an *identifier* (a `client_id`, an account name) as if it were a *secret* → pointless rotation and a false sense of exposure when it "leaks" (it was never a credential).
- Caching a per-user credential by a **user-supplied** key → one user fetches another's cached credential — the cross-user leak (the OBO cache-key gem, §1.9).
- Confusing authN with authZ → a perfectly valid token acting with the wrong permissions.
- Not checking a token's **audience/expiry** → token replay: a token minted for service A accepted by service B, or an expired one still honored.

Transferable mental model. Burn these in: **identity ≠ credential** (a name vs. the proof); **authN (who you are) vs. authZ (what you may do)**. Prove identity by *being* the workload; get a short-lived, signed token from the IdP; present it; the service verifies the **signature offline** (no callback to the IdP per request). Any secret that genuinely can't be eliminated lives in **one audited vault**, and the thing that unlocks that vault is — again — your workload identity. Once this model clicks, every platform's auth stack reads as the same story in a different accent.

PART 1 — GENERAL KNOWLEDGE

1.1 The secret-zero / bootstrapping-trust problem

Every auth story starts with a chicken-and-egg: **to get a secret you need a secret.** You want to read a database password out of a vault — but to open the vault you need *its* credential, which has to live somewhere: source code, a config file, an env var, a CI variable. That first stored credential is **secret zero**, and it is a permanent liability:

- it can **leak** (logged, screenshotted, pasted in a ticket),

- it can be **committed** to git and live forever in history,
- it **expires** and must be **rotated** (an operational treadmill),
- anyone who copies it can be you, anywhere, until it's revoked.

The whole modern direction — workload identity, managed identity, IAM roles — exists to **push secret zero down to the platform** so your code holds *no* long-lived secret at all. You can't leak what you don't have.

1.2 Identity vs credential (keep these distinct)

- **Identity / identifier** = a *name* for a principal: a username, an app's `client_id`, an AWS role ARN, a Kubernetes service-account name. **Not secret.** Publishing it gets an attacker nothing on its own.
- **Credential** = the *proof* you are that identity: a password, an API key, a private key/certificate, a signed token. **This is the secret.**

Confusing the two is a classic mistake: people treat a `client_id` or an account name as if leaking it were a breach. It isn't — an identifier only says *which* principal; you still need the credential (or, better, to *be* the workload, §1.4) to act as it.

1.3 Authentication vs authorization

Two separate questions, often run together:

- **AuthN (authentication) = who are you?** Proving identity (you present a credential; the system verifies it).
- **AuthZ (authorization) = what may you do?** Given a proven identity, checking permissions. This is where **roles / RBAC** live: an identity is *granted* roles ("read this vault", "query this cluster"), and each role bundles permissions.

A valid token can authenticate you perfectly and still be **denied** — authN passed, authZ failed. Least privilege lives entirely in authZ: give each identity the narrowest roles it needs.

1.4 Workload identity / Managed Identity (the general pattern)

The big idea that kills secret zero: **derive identity from BEING the workload, not from holding a secret.** Every major platform has a version:

Platform	Name
AWS	IAM Roles for EC2 / **IRSA** (IAM Roles for Service Accounts, on EKS)
GCP	**Workload Identity** (federation)
Azure	**Managed Identity**
Kubernetes	**Service Accounts** (projected tokens)

The common machinery:

- The platform **pre-registers** an identity and binds it to a running resource (a VM, a pod, a function).
- The resource asks a **local, on-machine endpoint** for a token. In cloud VMs this is the **instance metadata service** at the link-local address `169.254.169.254` — reachable *only from inside* that machine.
- **Identity comes from a platform-vouched local attestation:** the fact that a request reaches *this machine's own* metadata endpoint IS the proof — only that resource can reach it. There's no secret in your code to steal.

Crucial correctness point — who mints and signs the token: the resource does NOT make the token. The **identity provider** does. The metadata endpoint holds crypto material the *platform* planted (your app never sees it), uses it to prove the machine's identity to the IdP (Entra / AWS STS / Google), and the **IdP issues and cryptographically signs** the token. The downstream service trusts it because it carries the **IdP's unforgeable signature** (verified against the IdP's public key) — *not* because "the resource made it." Two pillars: (a) the local endpoint is only reachable from inside the machine (getting a token requires already being the resource); (b) the signing/proof material lives with the platform, not your code.

Analogy: a password is a **house key** — copyable, and whoever holds it is you. Workload identity is being **recognized by the security guard** — there's nothing to carry, steal, or rotate; you're vouched for by *being* who you are, in place.

1.5 OAuth2 / OIDC, tokens, and JWT signature verification

- **OAuth2** = the framework for *delegated authorization* (getting a scoped access token). **OIDC** = a thin identity layer on top of OAuth2 (adds an **ID token** describing *who* the user is).
- **Three token types: access token** (short-lived, presented to APIs to *do* things), **ID token** (says who the user is, OIDC), **refresh token** (long-lived, exchanged for new access tokens without re-login).
- **JWT (JSON Web Token)** = the common token format: `header.payload.signature`, base64url-encoded. The payload holds **claims** — including `oid` / `sub` (subject identity), `aud` (**audience** — which service this token is *for*), `scp` / `scope` (**scopes** — what it's allowed to do), `exp` (expiry).
- **Signature verification is the whole trust model:** the receiving service does NOT call the IdP per request. It fetches the IdP's **public keys** (once, cached) and checks the token's signature locally. Valid signature + correct `aud` + unexpired + right scopes → trusted. This is why tokens can be **short-lived** cheaply: verification is offline, so you re-issue often to limit blast radius if one leaks.
- **Audience matters:** a token minted *for* service A must not be accepted by service B — always check `aud`.

1.6 On-Behalf-Of / token exchange / delegation

Sometimes the app must act **as the user**, not **as itself**:

- **Act as the app** = the app's own identity and its (usually broad) permissions.
- **Act as the user** = use the *user's* permissions, so the app can only see what the user is authorized to see.

On-Behalf-Of (OBO) / token exchange is the pattern: the user's access token arrives at the app; the app proves *its own* identity and asks the IdP to **exchange** the user's token for a new token scoped to a downstream service — carrying the *user's* identity. The app never sees the user's password; it just relays and exchanges tokens.

Federated identity credential (FIC) is the modern way the app proves itself *during* that exchange **without a secret**: instead of an app secret/cert, the app presents a token from its *own* workload identity (§1.4) as the `client_assertion`. So even the OBO handshake is passwordless — trust is federated from the platform.

1.7 Secret managers (where unavoidable secrets live)

Workload identity removes *most* secrets, but not all — some legacy services still require a password, an API key, or a certificate. Those go in a **secret manager**, the one audited place secrets live:

Platform	Secret manager
Azure	**Key Vault**
AWS	**Secrets Manager** (and Parameter Store)
GCP	**Secret Manager**
any	**HashiCorp Vault**

The pattern: your workload identity (passwordless) **unlocks** the secret manager; the secret manager hands over the actual secret and **logs** the access. Access is gated by identity + RBAC, and every read is audited. Secret zero is gone because the *only* thing unlocking the vault is *being the workload*.

1.8 Certificates vs passwords vs tokens

Not everything speaks bearer tokens. Three credential shapes:

- **Password / shared secret** — simplest, most leakable; avoid for services.
- **Bearer token** (JWT) — short-lived, scoped, signed; the modern default. "Bearer" = whoever holds it can use it, so keep them short-lived and over TLS.
- **Certificate (mTLS)** — the client proves identity with a private key + cert during the TLS handshake itself, and the server does too (**mutual TLS**). Older or high-assurance systems often require a **cert instead of a bearer token** because they predate or don't trust the token IdP. The move: use workload identity to unlock the secret manager, pull the **cert**, and present *that* to the picky service — so you still store no secret in the app.

1.9 Caching credentials safely (a genuinely important security lesson)

Minting/exchanging tokens is expensive, so apps **cache** credentials (often with a TTL). The subtle, dangerous part is **the cache key**:

> ***The cache key MUST be a SERVER-VERIFIED identity — never a user-supplied string.***

If you key a per-user credential cache by the *raw token the user handed you* (or any user-controlled value), a crafted or colliding input could fetch **another user's** cached credential → **cross-user leak**. The fix: first *verify* the token (check the IdP signature, §1.5), extract the **verified** identity claim (e.g. the `oid` the IdP validated), and key the cache by **that**. TTL handles freshness; the *key choice* is purely about **cross-user isolation**. Principle, restated: **only trust server-verified identity as a cache key.** See [[kb-distributed-systems-patterns]].

PART 2 — WORKED EXAMPLE: the WCX get_azure_credential → MI / Key Vault / OBO stack

Auth in WCX is invisible plumbing: **`get_azure_credential()` sits under every client** (Kusto, Key Vault, Search — see [[kb-rag-retrieval]]). It only makes sense once you see the **secret-zero problem** (§1.1) it solves. One function, three modes; below is the "why passwordless works" chain so it's not a black box.

Three credential modes, one function (§1.2, §1.4, §1.6)

- **local** = personal login (chained).
- **prod-as-app** = Managed Identity (no secret).
- **prod-as-user** = OBO (the user's token exchanged for their own permissions).

`is_debug_mode()` = `DEBUG_MODE=true` **OR** no `AZURE_CLIENT_ID`.

- **Local** → `ChainedTokenCredential(Environment, AzureCli, VSCode, AzurePowerShell, InteractiveBrowser)` = "use whatever login the dev already has". A laptop can't use MI: it isn't an Azure resource — no fabric, no metadata endpoint, no provisioned identity (§1.4).
- **Prod** → `ManagedIdentityCredential(client_id=AZURE_CLIENT_ID)`, **no password anywhere**.

Same `get_azure_credential()` yields the dev's personal identity locally and the resource's MI in prod.

What Managed Identity physically IS (§1.4)

Not an IP or MAC (those are spoofable, network-level). Two pieces:

1. an **identity registered in Entra/AAD** — a **service principal** (a user account for an app); it gets an `oid` and can be granted roles (e.g. "read this Key Vault");
2. the resource proves it's that identity via a **private local metadata endpoint** (`169.254.169.254`, reachable *only from inside* that resource).

Identity comes from BEING the resource (only it can reach its own endpoint), not from holding a secret. The platform vouches from underneath. Analogy: password = **house key** (copyable); MI = recognized by the **security guard** (nothing to carry/steal/rotate).

`client_id` (`AZURE_CLIENT_ID`) is **NOT a secret** — it's a *name* for **which** assigned identity to use (a resource can have several). Knowing it gets an attacker nothing (they still can't *be* the resource). Identifier vs credential (§1.2), made concrete.

The MI token mechanism — Entra mints it, not the resource (§1.4 correctness point)

1. App asks the **local endpoint** (`169.254.169.254`, an on-machine program Azure runs) for a token.
2. Local endpoint requests the token from **Entra**, proving the machine's identity using crypto material the **platform planted** (app never sees it).
3. **Entra ISSUES + cryptographically SIGNS** the token.
4. Token flows back to the app.
5. App sends it to Kusto / Key Vault / etc.
6. That service trusts it because it carries **Entra's unforgeable signature** (verified via Entra's public key) — NOT because "the resource made it."

Two security pillars: (a) local endpoint only reachable from inside the machine (getting a token requires already *being* the resource); (b) proof material held by the endpoint, not app code (app holds no secret). **Hotel keycard analogy:** Entra = front desk (issues cards), local endpoint = in-room kiosk only you can reach, app = you, Kusto = amenity door that checks the front desk made the card.

Azure fabric = the platform (say "platform"): the datacenter software hosting all machines ("landlord"). Its role in MI is **STEP 0 / setup** — it created the local endpoint, planted the proof material, and recorded "this machine = this identity" (told Entra the assignment). You never touch it day-to-day; it explains *why* MI is trustworthy.

Key Vault = where unavoidable secrets live (§1.7)

MI removes MOST secrets, but some services need their own credential. Chain: the app uses MI (passwordless) to **unlock Key Vault** (`self._credential = get_azure_credential()`), and Key Vault hands over the secret (`get_secret` / `get_certificate_bytes`). Key Vault = Azure's dedicated, access-controlled, **audited** secret store; access is gated by identity (MI) + logged.

Why IcM needs a CERT (not MI) (§1.8): MI only works when the target trusts Entra/AAD tokens (Kusto, Key Vault, Azure Search do). **IcM authenticates with CERTIFICATES** (older system, no MI integration). Chain: MI unlocks Key Vault → Key Vault gives cert → app presents cert to IcM. MI is the master key that avoids storing *any* secret in the app, even for cert-based services.

On-Behalf-Of (OBO) = act AS the user, not the app (§1.6)

`get_obo_credential` . Why: sometimes the app must access data with the **user's own** permissions (a user only sees incidents they're authorized for), not the app's broad perms. Flow:

1. the user's access token arrives in the `Authorization` header;
2. the app proves ITS identity via MI using a **Federated Identity Credential** — `mi_credential.get_token("api://AzureADTokenExchange/.default")` as the `client_assertion` (passwordless proof, §1.6);
3. Azure **EXCHANGES** the user token → a service-scoped token, so the app calls Kusto **AS that user**.

Requires `OBO_TENANT_ID` / `OBO_CLIENT_ID` / `OBO_MI_CLIENT_ID` .

OBO CACHE KEY = verified `oid` , NOT the raw token (the security gem, §1.9): the OBO credential is cached (`TTLCache` , 10 min). It MUST be keyed by the **verified** `oid` (the AAD object id Azure validated), NEVER by the raw user token (user-controlled). A raw-token key → a crafted/colliding token could fetch **another user's** cached credential → cross-user leak. Principle: only trust **server-verified identity** as a cache key. (TTL handles freshness separately; the *key choice* is purely cross-user isolation.)

NonClosingCredential (adapter pattern)

The Kusto client closes its credential's HTTP transport on `.close()` . A **shared/cached** credential must NOT be torn down by one per-request client closing. `NonClosingCredential` wraps a shared credential, makes `.close()` a **no-op**, and delegates the rest (`__getattr__`). Same adapter pattern as the SearchClient wrapper ([[kb-rag-retrieval]]) — protects a shared resource from premature teardown.

EV2 (deploy layer, brief)

Express v2 = Microsoft's internal **safe-deployment** pipeline (staged regional rollouts, approvals). It's the "how it gets deployed" layer, **orthogonal to auth** — but it's also *how* the identities, `client_id` s, and Key Vault access above get provisioned into each region.

PART 3 — transferable takeaways (the interview/next-job version)

1. **Secret zero** is the root problem: to get a secret you need a secret. Every stored credential is a liability (leak/commit/expire/rotate) — push it down to the platform.
2. **Identity ≠ credential.** A `client_id` / ARN / account name is a *name* (not secret); the password/key/cert/token is the secret. Leaking an identifier is not a breach.
3. **AuthN (who you are) ≠ authZ (what you may do).** A valid token can still be denied by RBAC. Least privilege lives in authZ.
4. **Workload/Managed Identity** kills secret zero: identity from *being* the workload via a local metadata endpoint (`169.254.169.254`) — but the **IdP mints and signs** the token, and downstream services trust the **signature**, not the resource.

5. **Tokens are JWTs verified by signature offline** (`aud` / `scope` / `exp` checked); short-lived by design. OBO/token exchange lets an app act *as the user*; FIC makes even that handshake passwordless.
6. **Secret managers** (Key Vault / Secrets Manager / Vault) are the one audited place unavoidable secrets live, unlocked by workload identity. Cert-based/legacy services (mTLS) get their cert pulled from there.
7. **Cache credentials by SERVER-VERIFIED identity (the verified `oid`), never a user-supplied string** — else cross-user credential leakage. Only trust what you verified.